

MySQL Replication Concepts

What is Replication?

Replication is the process of **copying data from one MySQL server to another MySQL server automatically**.

- The main server is called the **Primary (Master)** server.
- The copied server is called the **Replica (Slave)** server.

Note: In modern MySQL documentation, the terms **Source** and **Replica** are preferred over **Master** and **Slave**.

Why Do We Use Replication?

- Backup of data
- High Availability
- Disaster Recovery
- Improve Read Performance
- Reduce Server Load

How Replication Works

Suppose we have:

Primary Server

Step 1

A user inserts data.

```
INSERT INTO Employees  
VALUES (1,'Kamalesh',50000);
```

Data is stored in the **Primary Server**.

Step 2

The Primary records the change in the **Binary Log (Binlog)**.

Binary Log

```
INSERT INTO Employees  
VALUES(1,'Kamalesh',50000);
```

Step 3

The Replica server reads the Binary Log.

Step 4

The Replica executes the same query.

Now both databases have the same data.

Example

Primary Server

Employees

ID	Name	Salary
1	Kamalesh	50000

Automatically copied to

Replica Server

Employees

ID	Name	Salary
1	Kamalesh	50000

Both tables become identical.

Types of Replication

1. Asynchronous Replication

Most commonly used.

Working

Client



Primary



Responds to client immediately



Replica receives data later

Example

User inserts data.

Primary immediately says

Insert Successful

Later

Replica receives the data.

Advantages

- Very fast
- Better performance

Disadvantage

If the Primary crashes before the Replica receives the data,
some recent data may be lost.

2. Semi-Synchronous Replication

Primary waits until **at least one Replica acknowledges** receiving the transaction.

Client

|



Primary

|

Waits for one Replica

|

Replica sends ACK

|

Primary replies to Client

Advantage

Safer than asynchronous replication.

Disadvantage

Slightly slower.

Types of Replication

1. Asynchronous Replication

Most commonly used.

Working

Client

|



Primary

|

|

Responds to client immediately

|



Replica receives data later

Example

User inserts data.

Primary immediately says

Insert Successful

Later

Replica receives the data.

Advantages

- Very fast
- Better performance

Disadvantage

If the Primary crashes before the Replica receives the data,
some recent data may be lost.

2. Semi-Synchronous Replication

Primary waits until **at least one Replica acknowledges** receiving the transaction.

Client

|



Primary

|

Waits for one Replica

|

Replica sends ACK

|

Primary replies to Client

Advantage

Safer than asynchronous replication.

Disadvantage

Slightly slower.

3. Group Replication

Several MySQL servers work together as one group.

Server1

Server2

Server3

If one server fails,
the others continue working.

Advantages

- High Availability
- Automatic Failover
- Fault Tolerance

Replication Modes

One Primary → One Replica

Primary

|



Replica

One Primary → Multiple Replicas

Replica1



|

Primary —+—▶ Replica2

|



Replica3

One Primary sends data to many Replicas.

The two MySQL servers are connected through a network (LAN or Internet)

Step 1: Create a Replication User on the Primary

The Primary creates a special user that the Replica can use to connect.

```
CREATE USER 'repl'@'%' IDENTIFIED BY 'password123';
```

```
GRANT REPLICATION SLAVE ON *.* TO 'repl'@'%';
```

```
FLUSH PRIVILEGES;
```

This means:

Username : repl

Password : password123

Step 2: Replica Connects to the Primary

The Replica is configured with the Primary's details.

It stores information like:

Host : 192.168.1.10

Port : 3306

Username : repl

Password : password123

Step 3: Replica Starts Replication

When replication starts:

Replica

|

| Connects to



Primary Server

It's similar to connecting to any MySQL server from MySQL Workbench.

Step 4: Connection Established

TCP/IP Connection

Replica -----> Primary
192.168.1.20 192.168.1.10
Username: repl
Password: password123

Once connected, the Replica begins reading the Primary's **Binary Log**.